

Binary Search Trees and Balanced Trees

BST, Skewed BST, AVL Tree, Red-Black Tree, B-Tree and B+ Tree

1 Binary Search Tree (BST)

1.1 Definition

A **Binary Search Tree (BST)** is a binary tree in which, for every node:

- All elements in the left subtree are smaller than the node.
- All elements in the right subtree are greater than the node.
- Both the left and right subtrees must themselves be Binary Search Trees.

Therefore, for a node containing value x :

$$\boxed{\text{Left Subtree} < x < \text{Right Subtree}}$$

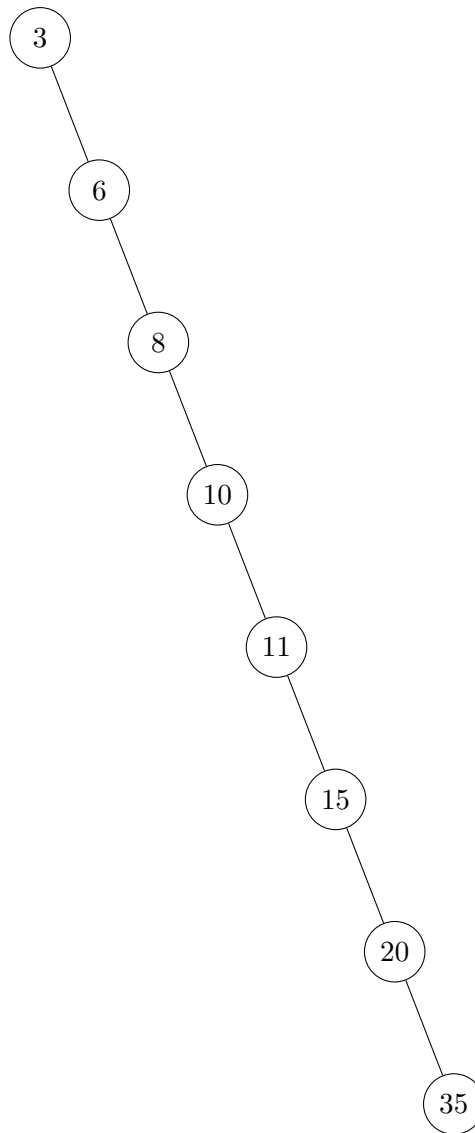
A normal BST does not automatically guarantee that the tree remains balanced.

1.2 BST Construction

Consider the elements:

3, 6, 8, 10, 11, 15, 20, 35

If inserted in this order, the BST becomes:



Because every new element is greater than the previous element, every new node is inserted to the right.

Thus, this BST is also a **skewed BST**.

2 Skewed Binary Search Tree

2.1 Definition

A **Skewed Binary Search Tree** is a BST in which every node has only one child, either a left child or a right child.

There are two types:

1. **Left-skewed tree**: every node has only a left child.
2. **Right-skewed tree**: every node has only a right child.

A skewed BST behaves similarly to a linked list.
Its height can become:

$$h = n - 1$$

where n is the number of nodes.

Therefore, searching, insertion and deletion can degrade to:

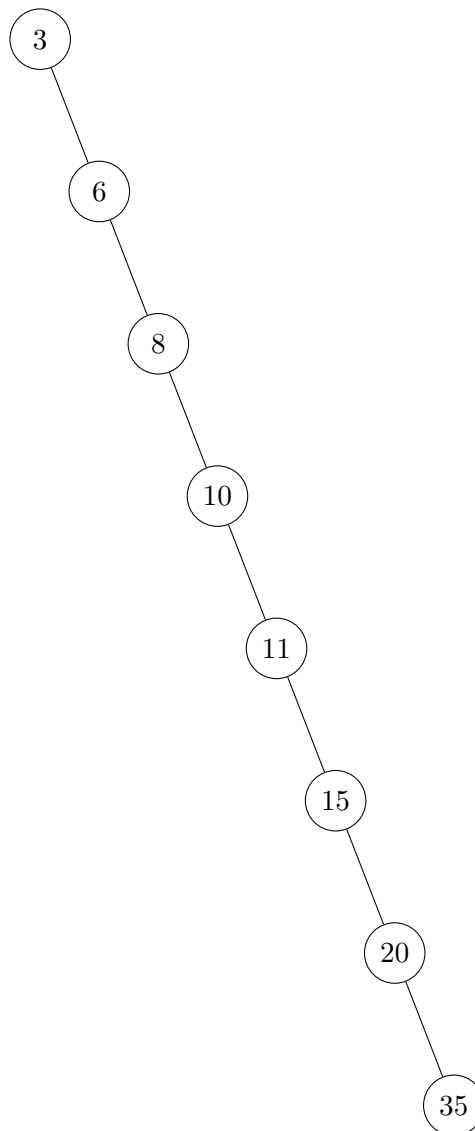
$$O(n)$$

2.2 Construction Using the Given Elements

Given:

3, 6, 8, 10, 11, 15, 20, 35

Since the elements are inserted in increasing order, every new element becomes the right child of the previous node.



This is a **right-skewed BST**.

2.3 Deletion of 11

The node 11 is a leaf node because it has no children.

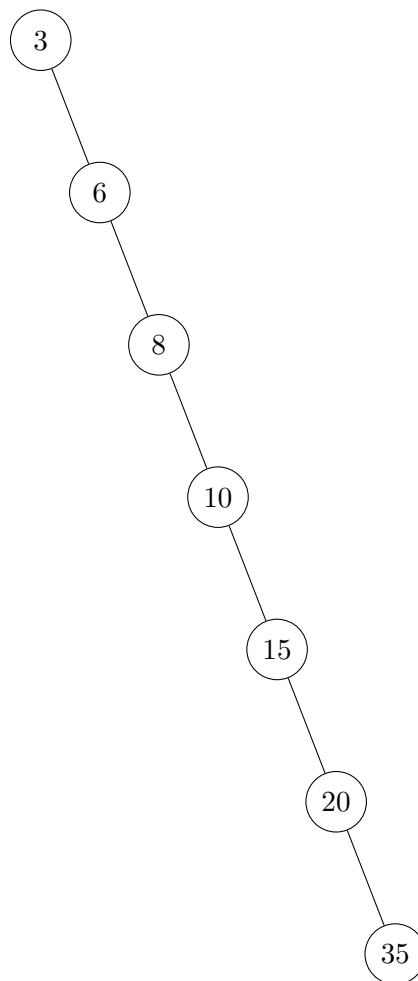
Therefore, deletion is simple:

$$10 \longrightarrow 11 \longrightarrow 15$$

becomes:

$$10 \longrightarrow 15$$

The resulting tree is:



Reason: Since 11 is a leaf, simply remove it.

2.4 Deletion of 20

After deleting 11, node 20 is also a leaf.

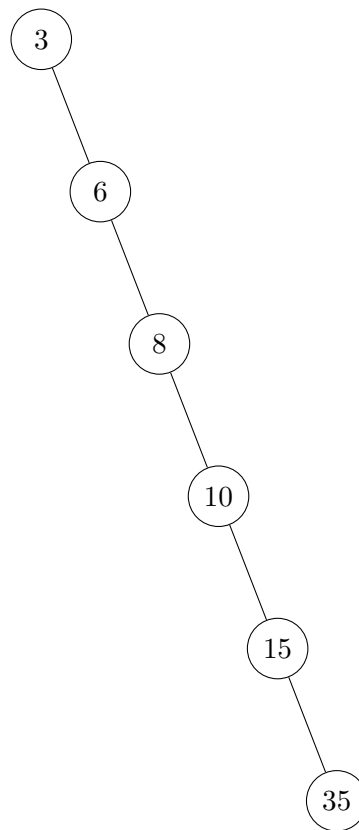
Therefore, remove 20:

$$15 \longrightarrow 20 \longrightarrow 35$$

becomes:

15 $\rightarrow$ 35

Result:



2.5 Deletion of 6

Node 6 has only one child, which is 8.

Therefore, when 6 is deleted, its child 8 takes its position.

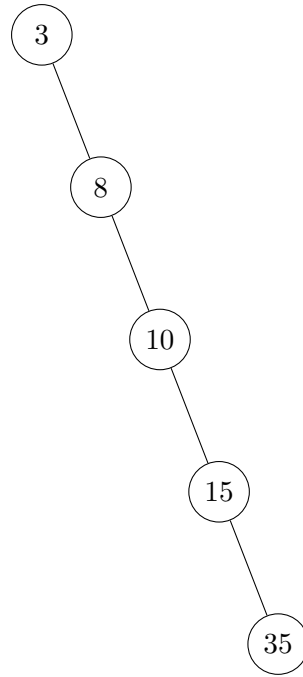
Before:

3 $\rightarrow$ 6 $\rightarrow$ 8

After deletion:

3 $\rightarrow$ 8

Result:



Important: A node with one child is deleted by connecting its parent directly to its child.

3 AVL Tree

3.1 Definition

An **AVL Tree** is a self-balancing Binary Search Tree in which the height difference between the left and right subtrees of every node is at most 1.

The balance factor of a node is:

$$BF = h(\text{Left Subtree}) - h(\text{Right Subtree})$$

For an AVL tree:

$$BF \in \{-1, 0, +1\}$$

If the balance factor becomes +2 or -2, the tree becomes unbalanced and rotations are performed.

The four types of AVL rotations are:

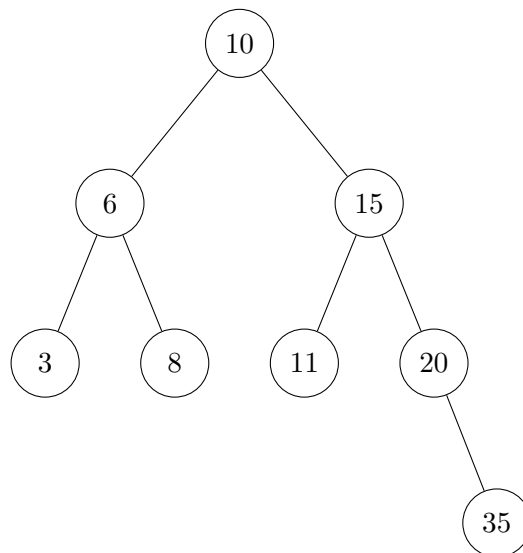
1. LL Rotation
2. RR Rotation
3. LR Rotation
4. RL Rotation

3.2 AVL Construction

The elements are inserted in the given order:

3, 6, 8, 10, 11, 15, 20, 35

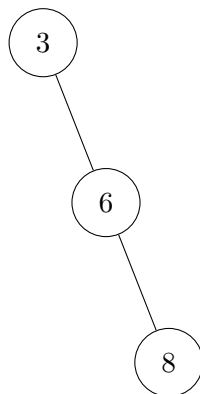
The final AVL tree is:



3.3 Important Rotations During Construction

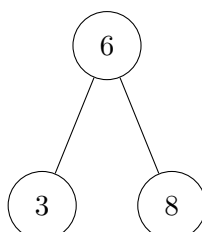
3.3.1 Insertion of 8

Initially:



Node 3 becomes unbalanced with an RR case.

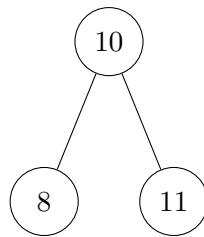
A left rotation at 3 gives:



3.3.2 Insertion of 11

The insertion creates an RR imbalance around node 8.

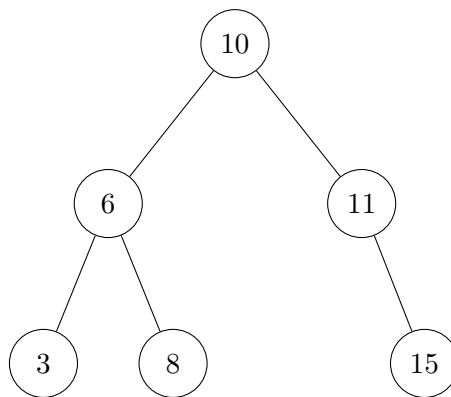
A left rotation gives:



3.3.3 Insertion of 15

The imbalance is corrected by a left rotation in the affected subtree.

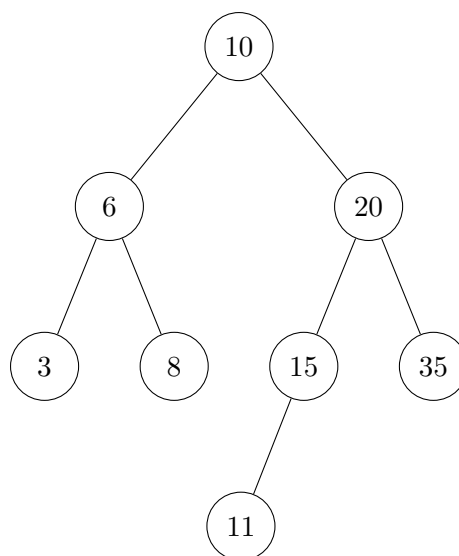
The tree becomes:



3.3.4 Insertion of 35

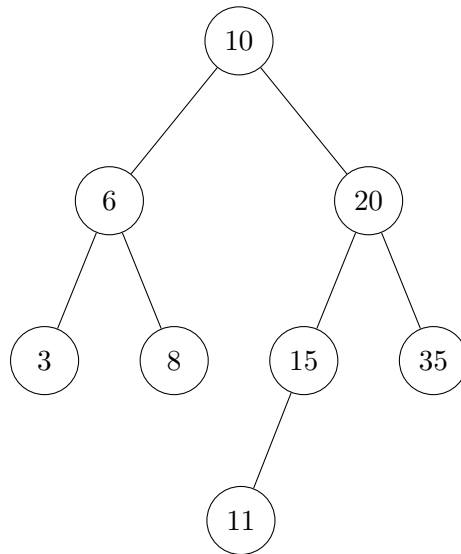
The right subtree becomes unbalanced and a left rotation is performed around node 15.

The final AVL tree is:

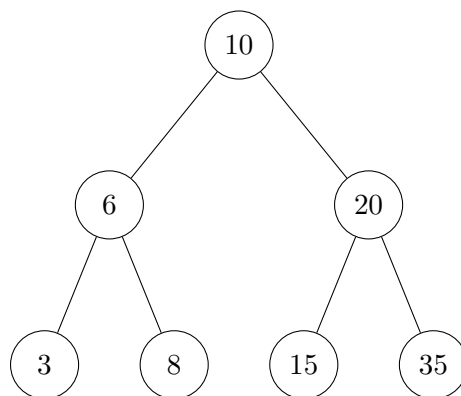


3.4 Deletion of 11

In the final AVL tree, 11 is a leaf:



Remove 11 directly.
The resulting tree is:



All balance factors remain within:

$$-1 \leq BF \leq 1$$

Therefore, no rotation is required.

3.5 Deletion of 20

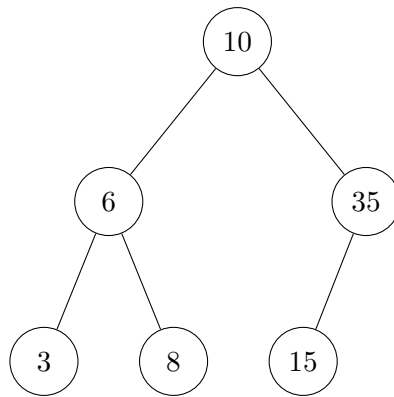
Node 20 has two children:

15 and 35

For deletion of a node with two children, we can replace it with its inorder successor.
The inorder successor of 20 is:

35

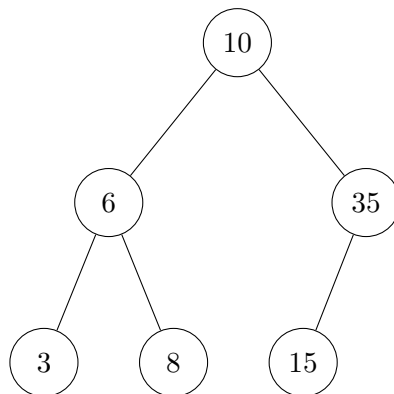
Replace 20 with 35 and remove the original 35.
Result:



No rotation is required because the resulting tree remains balanced.

3.6 Deletion of 6

After deleting 11 and 20, the relevant portion is:



Node 6 has two children:

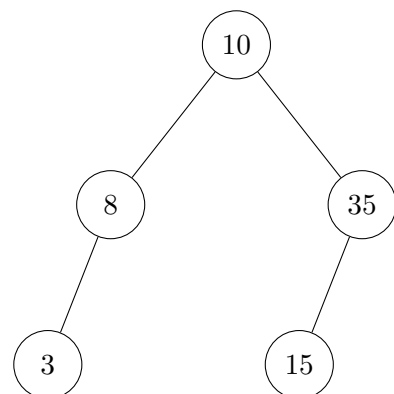
3 and 8

Its inorder successor is:

8

Replace 6 with 8 and remove the original 8.

The final AVL tree is:



Again, no rotation is necessary because the tree remains balanced.

4 Red-Black Tree

4.1 Definition

A **Red-Black Tree** is a self-balancing Binary Search Tree in which every node is assigned one of two colors:

RED or BLACK

A Red-Black Tree satisfies the following properties:

1. Every node is either red or black.
2. The root is always black.
3. Every NULL/NIL leaf is considered black.
4. A red node cannot have a red child.
5. Every path from a node to its descendant NIL leaves contains the same number of black nodes.

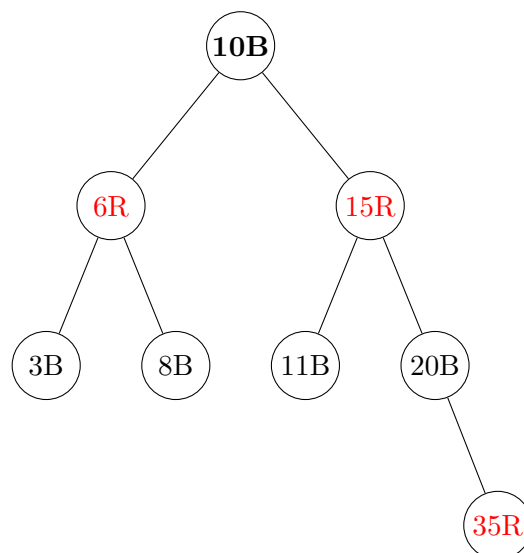
These properties ensure that the height of the tree remains approximately logarithmic.

4.2 Red-Black Tree Construction

The elements are inserted in the order:

3, 6, 8, 10, 11, 15, 20, 35

Using the standard Red-Black Tree insertion algorithm, the resulting tree is:



Here:

R = Red, B = Black

Thus:

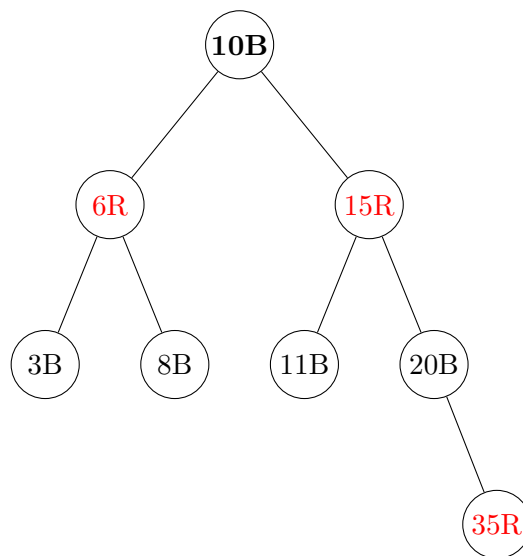
- 10 is black.
- 6 and 15 are red.
- 3, 8, 11 and 20 are black.
- 35 is red.

4.3 Deletion of 11

Node 11 is a black leaf.

Removing a black node can violate the black-height property.

Before deletion:

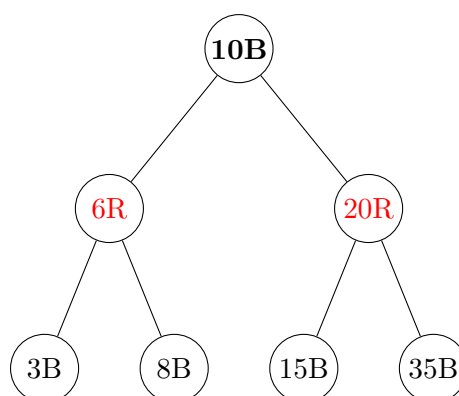


When 11 is removed, the resulting position is treated as a black NIL node.

The sibling of this NIL position is 20B, whose other child is 35R.

The Red-Black deletion fix-up recolors/rotates as necessary.

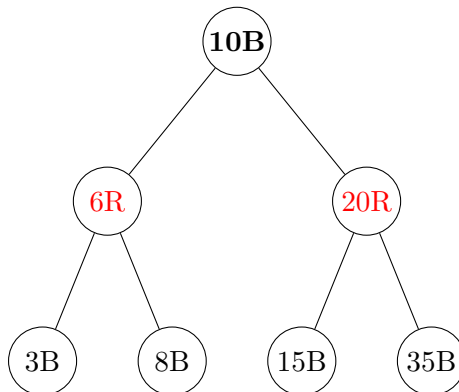
The resulting tree is:



The important result is that the black-height property is restored.

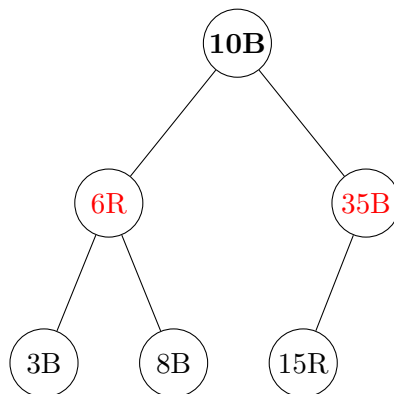
4.4 Deletion of 20

Node 20 is now a red leaf:



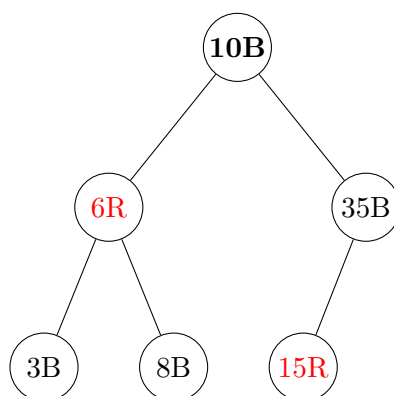
Since 20 is red and has no children, it can simply be removed. No black-height violation occurs.

Result:



4.5 Deletion of 6

After deleting 20, the tree is:



Node 6 has two children:

$3B$ and $8B$

Therefore, we use its inorder successor.

The inorder successor of 6 is:

8

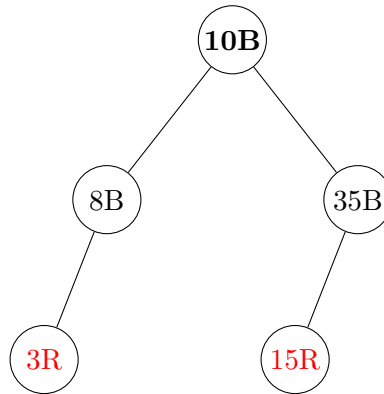
Replace 6 by 8.

The original 8 is a black leaf, so deleting it creates a temporary black-height deficiency.

The sibling of the deficient NIL node is 3B.

Since 3B has black children, it is recolored red during the deletion fix-up.

The final Red-Black Tree is:



Thus the final tree satisfies the Red-Black Tree properties again.

5 B-Tree

5.1 Theoretical Definition

A **B-Tree** is a height-balanced, multiway search tree designed to store large amounts of data efficiently, particularly in secondary storage such as disks.

Unlike a binary search tree, a B-Tree node can contain multiple keys and have multiple children.

For a B-Tree of order m :

- A node can have at most m children.
- A node can contain at most $m - 1$ keys.
- Except for the root, every internal node contains at least

$$\left\lceil \frac{m}{2} \right\rceil$$

children.

- All leaf nodes occur at the same level.
- Keys inside each node are stored in sorted order.

B-Trees are commonly used in databases and file systems because their large branching factor reduces the number of disk accesses.

6 B+ Tree

6.1 Theoretical Definition

A **B+ Tree** is a variation of the B-Tree in which all actual data records are stored only at the leaf level.

Internal nodes contain only keys used for searching and directing the search toward the appropriate leaf.

Important properties include:

- All actual records/data are stored in leaf nodes.
- Internal nodes contain keys and child pointers.
- All leaves are at the same level.
- Leaf nodes are generally linked together sequentially.
- The linked leaves make sequential and range searches efficient.

Therefore, a B+ Tree has two important levels of organization:

Internal Nodes → Search/Navigation

Leaf Nodes → Actual Data

The linked leaf nodes make operations such as range queries particularly efficient.

7 Summary of Deletions

Deleted Node	Skewed BST	AVL	Red-Black
11	Leaf deletion	Leaf deletion	Black leaf + fix-up
20	Leaf deletion	Two-child deletion	Red leaf deletion
6	One-child deletion	Two-child deletion	Two-child deletion + fix-up

Important Deletion Rules

Case 1: Node is a leaf

Simply remove the node

Case 2: Node has one child

Replace the node by its child

Case 3: Node has two children

Replace it with its inorder successor/predecessor

For AVL and Red-Black Trees, additional balancing operations may be required after deletion.